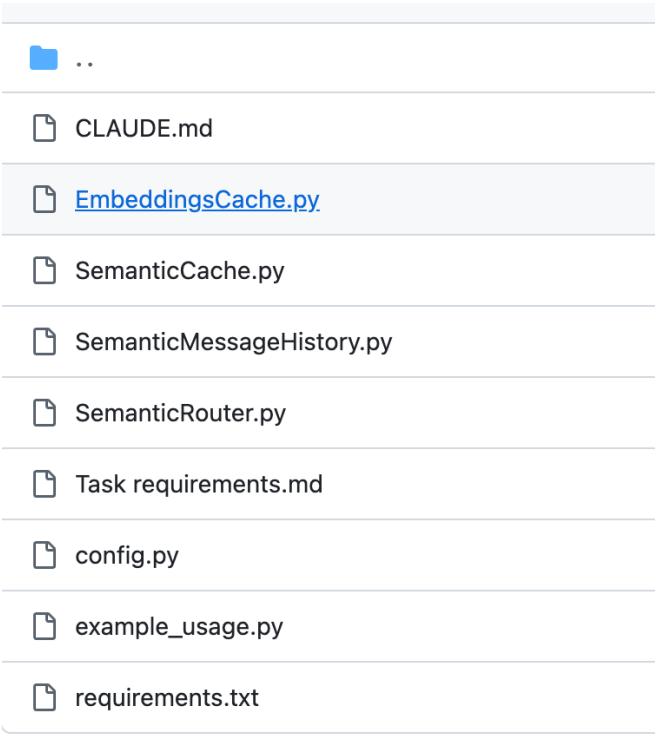
```python
from typing import Any, Dict, List, Optional

from redisvl.extensions.cache.llm import SemanticCache as _RedisVLSemanticCache

import c

class SemanticCache:
    """VLM-app semantic cache for prompt / response pairs."""

    def __init__(
        self,
        name: str = "vlm_llm_cache",
        distance_threshold: float = config.DEFAULT_DISTANCE_THRESHOLD,
        ttl: Optional[int] = config.DEFAULT_TTL,
        redis_url: Optional[str] = None,
        vectorizer=None,
        overwrite: bool = False,
    ) -> None:
        self.name = name
        self.vectorizer = config.resolve_vectorizer(vectorizer)

        kwargs: Dict[str, Any] = {
            "name": name,
            "distance_threshold": distance_threshold,
            "vectorizer": self.vectorizer,
            "redis_url": config.resolve_redis_url(redis_url),
            "overwrite": overwrite,
        }
        if ttl is not None:
```

Code view is read-only. Switch to the editor.

错误实现，需要自己实现，而不是封装已有的实现。



错误实现，没有目录结构

```python
def _hash_key(content: str | bytes, model_name: str) -> str:
    """Generate a deterministic hash from content and model name."""
    if isinstance(content, bytes):
        content = content.hex()
    raw = f"{content}:{model_name}"
    return hashlib.sha256(raw.encode()).hexdigest()[:32]


@dataclass
class CacheEntry:
    """A single cached embedding entry."""
    entry_id: str
    content: str
    model_name: str
    embedding: list[float]
    inserted_at: float = field(default_factory=time.time)
    metadata: dict[str, Any] | None = None


class EmbeddingsCache:
    """Cache embeddings by content+model to avoid duplicate API calls.

    Usage:
        cache = EmbeddingsCache("embeds", ttl=3600, redis_url="redis://localhost:6379")
        cache.set("hello world", "text-embedding-3", [0.1, 0.2, ...])
        emb = cache.get("hello world", "text-embedding-3")  # -> list[float] or None
        exists = cache.exists("hello world", "text-embedding-3")
    """

```

错误实现，没有使用 pydantic，数据定义和 class 不应该卸载一个文件中

```python
"""向量相似度工具。"""

import math
from typing import List


def cosine_similarity(a: List[float], b: List[float]) -> float:
    if len(a) != len(b) or not a:
        return 0.0
    dot = sum(x * y for x, y in zip(a, b))
    norm_a = math.sqrt(sum(x * x for x in a))
    norm_b = math.sqrt(sum(y * y for y in b))
    if norm_a == 0.0 or norm_b == 0.0:
        return 0.0
    return dot / (norm_a * norm_b)
```

错误实现，为什么不用 numpy?